

Complexity

复杂度简介

时间复杂度和空间复杂度是衡量一个算法效率的重要标准。

基本操作数

同一个算法在不同的计算机上运行的速度会有一定的差别，并且实际运行速度难以在理论上进行计算，实际去测量又比较麻烦，所以我们通常考虑的不是算法运行的实际用时，而是算法运行所需要进行的基本操作的数量。

在普通的计算机上，加减乘除、访问变量（基本数据类型的变量，下同）、给变量赋值等都可以看作基本操作。

对基本操作的计数或是估测可以作为评判算法用时的指标。

时间复杂度

定义

衡量一个算法的快慢，一定要考虑数据规模的大小。所谓数据规模，一般指输入的数字个数、输入中给出的图的点数与边数等等。一般来说，数据规模越大，算法的用时就越长。而在算法竞赛中，我们衡量一个算法的效率时，最重要的不是看它在某个数据规模下的用时，而是看它的用时随数据规模而增长的趋势，即 **时间复杂度**。

引入

考虑用时随数据规模变化的趋势的主要原因有以下几点：

- 现代计算机每秒可以处理数亿乃至更多次基本运算，因此我们处理的数据规模通常很大。如果算法 A 在规模为 n 的数据上用时为 $T_A(n)$ ，而算法 B 在规模为 n 的数据上用时为 $T_B(n)$ ，在数据规模小于 n_0 时算法 B 用时更短，但在一秒钟内算法 A 可以处理数百万规模的数据，而算法 B 只能处理数万规模的数据。在允许算法执行时间更久时，时间复杂度对可处理数据规模的影响就会更加明显，远大于同一数据规模下用时的影响。
- 我们采用基本操作数来表示算法的用时，而不同的基本操作实际用时是不同的，例如加法的用时远小于除法的用时。计算时间复杂度而忽略不同基本操作之间的区别以及一次基本操作与十次基本操作之间的区别，可以消除基本操作间用时不同的影响。

当然，算法的运行用时并非完全由输入规模决定，而是也与输入的内容相关。所以，时间复杂度又分为几种，例如：

- 最坏时间复杂度，即每个输入规模下用时最长的输入对应的时间复杂度。在算法竞赛中，由于输入可以在给定的数据范围内任意给定，我们为保证算法能够通过某个数据范围内的任何数据，一般考虑最坏时间复杂度。
- 平均（期望）时间复杂度，即每个输入规模下所有可能输入对应用时的平均值的复杂度（随机输入下期望用时的复杂度）。

所谓「用时随数据规模而增长的趋势」是一个模糊的概念，我们需要借助下文所介绍的 **渐进符号** 来形式化地表示时间复杂度。

渐进符号的定义

渐进符号是函数的阶的规范描述。简单来说，渐进符号忽略了一个函数中增长较慢的部分以及各项的系数（在时间复杂度相关分析中，系数一般被称作「常数」），而保留了可以用来表明该函数增长趋势的重要部分。

一个简单的记忆方法是，含等于（非严格）用大写，不含等于（严格）用小写，相等是，小于是，大于是。大 和 小 原本是希腊字母 Omicron，由于字形相同，也可以理解为拉丁字母的大和小。

在英文中，词根「micro-」和「mega-」常用于表示 10 的负六次方（百万分之一）和六次方（百万），也表示「小」和「大」。小和大也是希腊字母 Omicron 和 Omega 常表示的含义。

大 O 符号

对于函数 f 和 g ， $f = O(g)$ ，当且仅当，使得

也就是说，如果函数 f ，那么我们能找到两个正数 c 使得 f 被 cg 夹在中间。

例如，这里的 $f(x) = 2x^2 + 3x + 4$ 可以分别是 $2x^2$ ，这里的 $g(x) = x^2$ 可以分别是 x^2 。

大 Ω 符号

符号同时给了我们一个函数的上下界，如果只知道一个函数的渐进上界而不知道其渐进下界，可以使用 Ω 符号。 $f = \Omega(g)$ ，当且仅当，使得

研究时间复杂度时通常会使用 Θ 符号，因为我们关注的通常是程序用时的上界，而不关心其用时的下界。

需要注意的是，这里的「上界」和「下界」是对于函数的变化趋势而言的，而不是对算法而言的。算法用时的上界对应的是「最坏时间复杂度」而非大 O 记号。所以，使用 Θ 记号表示最坏时间复杂度是完全可行的，甚至可以说比 O 更加精确，而使用 Θ 记号的主要原因，一是我们有时只能证明时间复杂度的上界而无法证明其下界（这种情况一般出现在较为复杂的算法以及复杂度分析），二是 在电脑上输入更方便一些。

大 ω 符号

同样的，我们使用 ω 符号来描述一个函数的渐进下界。 $f = \omega(g)$ ，当且仅当，使得

小 o 符号

如果说 $f = o(g)$ 符号相当于小于等于号，那么 $f = O(g)$ 符号就相当于小于号。

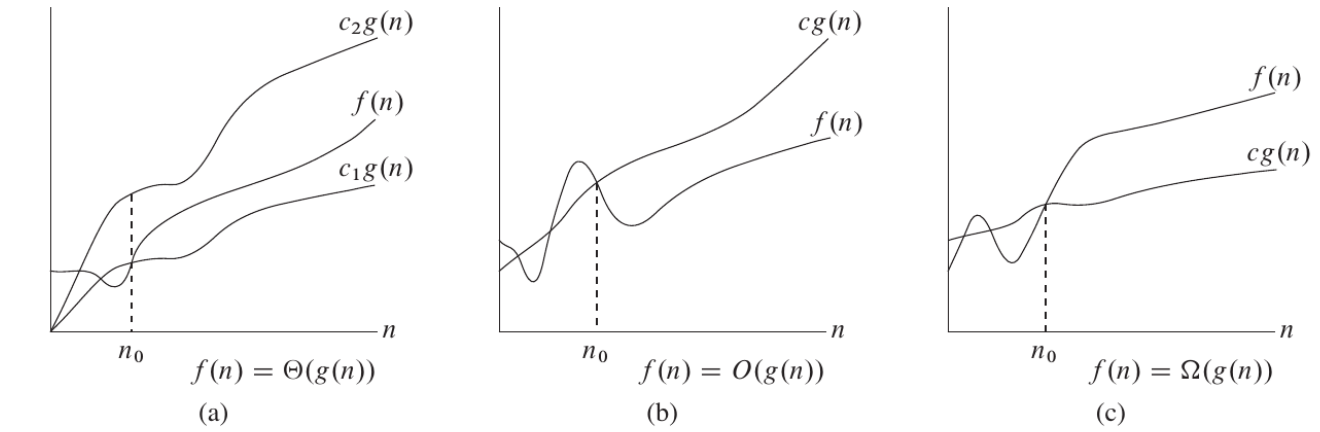
小 o 符号大量应用于数学分析中，函数在某点处的泰勒展开式拥有皮亚诺余项，使用小 o 符号表示严格小于，从而进行等价无穷小的渐进分析。

$f = o(g)$ ，当且仅当对于任意给定的正数 ϵ ，使得 $f < \epsilon g$ 。

小 Ω 符号

如果说 $f = \Omega(g)$ 符号相当于大于等于号，那么 $f = \omega(g)$ 符号就相当于大于号。

$f = \Omega(g)$ ，当且仅当对于任意给定的正数 ϵ ，使得 $f > \epsilon g$ 。



常见性质

-
-
-
- 由换底公式可以得知，任何对数函数无论底数为何，都具有相同的增长率，因此渐进时间复杂度中对数的底数一般省略不写。

简单的时间复杂度计算的例子

for 循环

  
C++PythonJava

```
1 int n, m;
2 std::cin >> n >> m;
3 for (int i = 0; i < n; ++i) {
4     for (int j = 0; j < n; ++j) {
5         for (int k = 0; k < m; ++k) {
6             std::cout << "hello world\n";
7         }
8     }
9 }
```

```
1 n = int(input())
2 m = int(input())
3 for i in range(0, n):
4     for j in range(0, n):
5         for k in range(0, m):
6             print("hello world")
```

```
1 int n, m;
2 n = input.nextInt();
3 m = input.nextInt();
4 for (int i = 0; i < n; ++i) {
5     for (int j = 0; j < n; ++j) {
6         for (int k = 0; k < m; ++k) {
7             System.out.println("hello world");
8         }
9     }
10 }
```

如果以输入的数值 n 和 m 的大小作为数据规模，则上面这段代码的时间复杂度为 $O(n^2m)$ 。

DFS

在对一张 n 个点 m 条边的图进行 [DFS](#) 时，由于每个节点和每条边都只会被访问常数次，复杂度为 $O(n+m)$ 。

哪些量是常量？

当我们要进行若干次操作时，如何判断这若干次操作是否影响时间复杂度呢？例如：

  
C++PythonJava

```
1 constexpr int N = 100000;
2 for (int i = 0; i < N; ++i) {
3     std::cout << "hello world\n";
4 }
```

```
1 N = 100000
2 for i in range(0, N):
3     print("hello world")
```

```
1 final int N = 100000;
2 for (int i = 0; i < N; ++i) {
3     System.out.println("hello world");
4 }
```

如果 N 的大小不被看作输入规模，那么这段代码的时间复杂度就是 $O(1)$ 。

进行时间复杂度计算时，哪些变量被视作输入规模是很重要的，而所有和输入规模无关的量都被视作常量，计算复杂度时可当作 $O(1)$ 来处理。

需要注意的是，在进行时间复杂度相关的理论性讨论时，「算法能够解决任何规模的问题」是一个基本假设（当然，在实际中，由于时间和存储空间有限，无法解决规模过大的问题）。因此，能在常量时间内解决数据规模有限的问题（例如，对于数据范围内的每个可能输入预先计算出答案）并不能使一个算法的时间复杂度变为 $O(1)$ 。

主定理 (Master Theorem)

我们可以使用 Master Theorem 来快速求得关于递归算法的复杂度。Master Theorem 递推关系式如下

那么

需要注意的是，这里的第二种情况还需要满足 regularity condition, 即，for some constant ϵ and sufficiently large n 。

证明思路是是将规模为 n 的问题，分解为 a 个规模为 $\frac{n}{b}$ 的问题，然后依次合并，直到合并到最高层。每一次合并子问题，都需要花费 $O(n^d)$ 的时间。

► 证明

下面举几个例子来说明主定理如何使用。

1. $a=1, b=1, d=0$ ，那么， $\frac{n}{b} = n$ 可以取值在 n 之间，从而满足第一种情况，所以 $T(n) = O(n^0) = O(1)$ 。
2. $a=1, b=2, d=0$ ，那么， $\frac{n}{b} = \frac{n}{2}$ 可以取值在 $\frac{n}{2}$ 之间，从而满足第二种情况，所以 $T(n) = O(n^0 \log n) = O(\log n)$ 。
3. $a=1, b=2, d=1$ ，那么， $\frac{n}{b} = \frac{n}{2}$ 可以取值为 $\frac{n}{2}$ ，从而满足第三种情况，所以 $T(n) = O(n^1) = O(n)$ 。
4. $a=2, b=2, d=0$ ，那么， $\frac{n}{b} = \frac{n}{2}$ 可以取值为 $\frac{n}{2}$ ，从而满足第三种情况，所以 $T(n) = O(n^0) = O(1)$ 。

均摊复杂度

详情可见 [均摊复杂度](#)。

空间复杂度

类似地，算法所使用的空间随输入规模变化的趋势可以用 **空间复杂度** 来衡量。

计算复杂性

本文主要从算法分析的角度对复杂度进行了介绍，如果有兴趣的话可以在 [计算复杂性](#) 进行更深入的了解。

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[iamtwz](#), [linehk](#), [ouuan](#), [abc1763613206](#), [aquastripe](#), [Back1ght](#), [blackwhitetony](#), [CCXXXI](#), [chinggg](#), [ChungZH](#), [Enonya](#), [Enter-tainer](#), [Great-designer](#), [GreyTigerOler](#), [GuanghaoYe](#), [heshuyue](#), [lr1d](#), [isdanni](#), [ksyx](#), [Marcythm](#), [Menci](#), [mgt](#), [nullnan](#), [onelittlechildawa](#), [partychicken](#), [persdre](#), [Persdre](#), [shawlleym](#), [shuzhouliu](#), [sshwy](#), [TianyiQ](#), [Tiphereth-A](#), [wr786](#), [Xeonacid](#), [YHN-ice](#), [yyyu-star](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用